



MODUL TEORI

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.

Dosen Pengampuh Matkul IBD

Contents

Materi 4 Subquery, Common Table Expression (CTE), dan Window Function.....	3
Tujuan Pembelajaran.....	3
A. KONSEP DASAR SUBQUERY	3
B. JENIS-JENIS SUBQUERY	4
C. SUBQUERY BERDASARKAN LOKASI PENGGUNAAN	6
D. CORRELATED SUBQUERY	8
E. COMMON TABLE EXPRESSION	9
F. CTE REKURSIF.....	11
G. KONSEP DASAR WINDOW FUNCTION	12
H. KLAUSA OVER.....	14
I. ROW_NUMBER	16
J. RANK DAN DENSE_RANK	17
K. NTILE	19
L. LEAD DAN LAG	20

Materi 4

Subquery, Common Table Expression (CTE), dan Window Function

Tujuan Pembelajaran

Setelah mempelajari materi ini, mahasiswa diharapkan mampu:

1. Membuat query di dalam query menggunakan Subquery.
2. Membentuk Result Set sementara menggunakan CTE.
3. Memberikan nomor urut dan peringkat menggunakan Window Function.
4. Membagi data menjadi beberapa kelompok menggunakan NTILE.
5. Membandingkan data saat ini dengan data sebelumnya menggunakan LAG.
6. Mengakses data berikutnya menggunakan LEAD

A. KONSEP DASAR SUBQUERY

1. *Pengertian Subquery*

Subquery adalah query yang ditempatkan di dalam query lain. Subquery menghasilkan sebuah nilai atau Result Set yang kemudian digunakan oleh query utama.

Subquery juga dikenal sebagai:

- inner query;
- nested query;
- query bersarang.

Query yang menggunakan hasil Subquery disebut:

- outer query;
- main query;
- query utama.

2. *Cara Kerja Subquery*

Secara konseptual, proses Subquery berlangsung sebagai berikut:

- SQL Server membaca query secara keseluruhan.
- SQL Server mengidentifikasi subquery.
- Subquery dievaluasi sesuai kebutuhan query utama.
- Hasil subquery digunakan sebagai nilai, daftar nilai, atau sumber data.
- Query utama memproses hasil tersebut.
- SQL Server menghasilkan result set akhir.

Sebagai contoh konseptual, manajemen ingin mengetahui produk yang harganya di atas rata-rata.

Langkah 1

Hitung rata-rata harga seluruh produk

Langkah 2

Bandingkan harga setiap produk dengan rata-rata

Langkah 3

Tampilkan produk yang memenuhi kondisi

Perhitungan rata-rata dapat dilakukan dengan subquery, sedangkan penyaringan produk dilakukan oleh query utama.

3. *Komponen Subquery*

Subquery memiliki dua komponen utama.

- Query utama: Query utama menentukan informasi akhir yang akan ditampilkan.
- Query bagian dalam: Query bagian dalam menghasilkan informasi yang dibutuhkan oleh query utama.

B. JENIS-JENIS SUBQUERY

1. *Klasifikasi Subquery*

Subquery dapat diklasifikasikan berdasarkan:

- jumlah nilai yang dihasilkan;
- jumlah kolom yang dihasilkan;
- hubungan dengan query utama;
- Lokasi penggunaannya.

Berdasarkan jumlah hasil, Subquery terdiri atas:

- Scalar Subquery;
- Single-Row Subquery;
- Multiple-Row Subquery;
- Multiple-Column Subquery.

Berdasarkan hubungan dengan query utama, Subquery terdiri atas:

- Non-Correlated Subquery;
- Correlated Subquery.

2. *Scalar Subquery*

Scalar Subquery adalah Subquery yang menghasilkan tepat satu nilai.

Nilai tersebut dapat berupa:

- angka;
- teks;
- tanggal;
- nilai agregasi;
- nilai identitas tertentu.

Contoh kebutuhan:

- harga rata-rata seluruh produk;
- jumlah pelanggan;
- harga layanan tertinggi;
- tanggal penjualan terbaru;
- Total stok seluruh produk.

Karakteristik Scalar Subquery

- hanya menghasilkan satu kolom;
- maksimal menghasilkan satu baris;
- dapat digunakan sebagai nilai pembandingan;
- dapat digunakan sebagai kolom hasil;
- Sering menggunakan fungsi agregasi.

3. *Single-Row Subquery*

Single-Row subquery menghasilkan satu baris data. Baris tersebut dapat memiliki satu atau lebih kolom, tergantung pada penggunaannya.

Contoh kebutuhan:

- mendapatkan data produk termahal;
- mendapatkan tanggal transaksi terakhir;
- menemukan nilai penjualan terbesar;
- Memperoleh satu catatan audit terbaru.

Single-Row Subquery cocok digunakan ketika query utama membutuhkan satu baris acuan.

4. *Multiple-Row Subquery*

Multiple-Row Subquery menghasilkan lebih dari satu baris.

Contoh kebutuhan:

- daftar pelanggan yang pernah melakukan penjualan;
- daftar produk yang pernah terjual;
- daftar pegawai yang pernah menangani reservasi;

- daftar kategori yang memiliki produk;
- daftar hewan yang telah memperoleh layanan.

Query utama harus menggunakan cara perbandingan yang mampu menangani banyak nilai.

5. *Multiple-Column Subquery*

Multiple-Column Subquery menghasilkan lebih dari satu kolom.

Contohnya adalah Subquery yang menghasilkan pasangan:

KodeProduk | Harga

atau:

KodePelanggan | TanggalTransaksi

Jenis subquery ini digunakan ketika pencocokan harus mempertimbangkan beberapa atribut secara bersamaan.

6. *Non-Correlated Subquery*

Non-Correlated subquery dapat dijalankan secara mandiri karena tidak bergantung pada data dari query utama.

Contoh kebutuhan:

- menghitung harga rata-rata produk;
- menentukan harga tertinggi;
- memperoleh daftar pelanggan yang bertransaksi;
- menemukan tanggal penjualan terbaru.

7. *Correlated Subquery*

Correlated Subquery bergantung pada baris yang sedang diproses oleh query utama.

C. SUBQUERY BERDASARKAN LOKASI PENGGUNAAN

1. *Subquery sebagai Kondisi Penyaringan*

Subquery paling umum digunakan pada bagian penyaringan. Subquery menghasilkan:

- satu nilai pembanding;
- sekumpulan nilai;
- informasi tentang keberadaan data;
- kondisi yang harus dipenuhi.

Contoh kebutuhan DB_Petshop:

- produk dengan harga di atas rata-rata;
- pelanggan yang pernah melakukan penjualan;
- produk yang belum pernah terjual;

- layanan yang pernah dipesan;
- pegawai yang menangani reservasi.

2. *Subquery sebagai Kolom Hasil*

Subquery dapat digunakan untuk menghasilkan kolom tambahan dalam result set.

Misalnya, setiap pelanggan ditampilkan bersama:

- jumlah transaksi;
- jumlah hewan;
- jumlah reservasi;
- transaksi terakhir;
- total pembelian.

3. *Subquery sebagai Sumber Data*

Subquery dapat menghasilkan result set yang digunakan seperti tabel sementara.

Result Set tersebut kemudian dapat:

- diberi nama alias;
- digabungkan dengan tabel lain;
- disaring kembali;
- dikelompokkan;
- diurutkan.

Subquery yang digunakan sebagai sumber data sering disebut sebagai derived table.

4. *Subquery pada Proses Pengelompokan*

Subquery dapat membantu proses pengelompokan pada tingkat tertentu.

Contohnya:

- menghitung total pembelian setiap pelanggan;
- menghitung rata-rata total pembelian pelanggan;
- menampilkan pelanggan dengan total di atas rata-rata.

Proses tersebut memerlukan lebih dari satu tingkat perhitungan.

5. *Subquery untuk Pemeriksaan Keberadaan Data*

Subquery dapat digunakan untuk menjawab pertanyaan seperti:

- Apakah pelanggan memiliki transaksi?
- Apakah produk pernah terjual?
- Apakah pegawai pernah menangani reservasi?
- Apakah layanan pernah digunakan?
- Apakah produk memiliki catatan AuditStok?

Dalam kondisi tersebut, hal terpenting bukan nilai yang dihasilkan, melainkan keberadaan baris yang memenuhi kondisi tersebut.

D. CORRELATED SUBQUERY

1. *Pengertian Correlated Subquery*

Correlated Subquery adalah Subquery yang menggunakan nilai dari baris yang sedang diproses oleh query utama. Subquery tersebut tidak dapat dievaluasi secara independen karena membutuhkan referensi dari query lain.

2. *Cara Kerja Correlated Subquery*

Secara konseptual:

- Query utama membaca satu baris.
- Nilai dari baris tersebut diteruskan ke subquery.
- Subquery melakukan evaluasi berdasarkan nilai itu.
- Hasil subquery dikembalikan ke query utama.
- Query utama menentukan apakah baris ditampilkan.
- Proses diulang untuk baris berikutnya.

3. *Contoh Kebutuhan pada DB_Petshop*

Correlated Subquery dapat digunakan untuk:

- menampilkan produk dengan harga tertinggi pada kategorinya;
- mencari transaksi terakhir setiap pelanggan;
- mencari audit terbaru setiap produk;
- menemukan pelanggan yang memiliki lebih dari satu hewan;
- mencari pegawai dengan reservasi terbanyak dalam kelompok tertentu;
- membandingkan harga produk dengan rata-rata kategori yang sama;
- mencari produk yang stoknya di bawah rata-rata kategori.

4. *Keunggulan Correlated Subquery*

- mampu mengevaluasi kondisi spesifik untuk setiap baris;
- cocok untuk perbandingan dalam kelompok;
- efektif untuk pemeriksaan keberadaan hubungan;
- dapat mengekspresikan logika bisnis secara langsung.

5. *Keterbatasan Correlated Subquery*

- berpotensi dievaluasi berulang kali;
- dapat lebih lambat pada tabel besar;
- lebih sulit dibaca;

- sulit dioptimalkan jika struktur terlalu kompleks;
- Beberapa kasus lebih baik diselesaikan dengan JOIN, CTE, atau Window Function.

E. COMMON TABLE EXPRESSION

1. *Pengertian CTE*

Common Table Expression, disingkat CTE, adalah result set sementara yang diberi nama dan dapat digunakan dalam pernyataan SQL yang mengikutinya. CTE bukan tabel fisik dan tidak menyimpan data secara permanen. CTE dapat dipahami sebagai cara untuk membagi query kompleks menjadi beberapa bagian yang lebih mudah dibaca.

2. *Karakteristik CTE*

CTE memiliki karakteristik berikut:

- Hanya berlaku untuk pernyataan berikutnya.
- Tidak disimpan secara permanen.
- Dapat diberi nama yang bermakna.
- Dapat direferensikan seperti tabel.
- Dapat digunakan untuk operasi pembacaan maupun modifikasi tertentu.
- Dapat digabungkan dengan CTE lain.
- Dapat digunakan untuk proses rekursif.
- Meningkatkan keterbacaan query kompleks.

3. *Tujuan Penggunaan CTE*

- Menyederhanakan query kompleks
- Query besar dapat dibagi menjadi tahapan yang logis.
- Menghindari pengulangan Subquery
- Result Set tertentu dapat diberi nama agar struktur query lebih jelas.
- Membentuk proses pengolahan bertahap

Misalnya:

- menghitung total pembelian;
- menentukan peringkat;
- menyaring pelanggan tertentu.

Menangani data hierarkis

CTE rekursif dapat digunakan untuk struktur induk-anak.

Meningkatkan dokumentasi query

Nama CTE dapat menjelaskan fungsi Result Set yang dibentuk

4. *CTE Nonrekursif*

CTE nonrekursif adalah CTE yang tidak memanggil dirinya sendiri. Jenis ini digunakan untuk:

- menyederhanakan query;
- membentuk data agregat;
- mempersiapkan data sebelum JOIN;
- membuat proses bertahap;
- menggunakan Window Function;
- menyaring hasil peringkat.

Contoh kebutuhan DB_Petshop:

- total pembelian setiap pelanggan;
- total penjualan setiap produk;
- jumlah reservasi setiap layanan;
- riwayat AuditStok;
- nilai transaksi per hari.

5. *Ruang Lingkup CTE*

CTE hanya dapat digunakan oleh satu pernyataan yang langsung mengikuti definisinya.

Artinya, CTE:

- tidak menjadi objek database permanen;
- tidak tersedia untuk query berikutnya;
- tidak dapat dipanggil dari sesi lain;
- harus didefinisikan kembali jika ingin digunakan lagi.

6. *Keuntungan CTE*

- query lebih mudah dibaca;
- logika dapat dipisahkan menjadi tahapan;
- mendukung rekursi;
- mempermudah pengujian logika;
- nama CTE dapat menjelaskan tujuan data;
- cocok dikombinasikan dengan Window Function.

7. *Keterbatasan CTE*

- tidak menyimpan hasil secara permanen;
- tidak selalu meningkatkan performa;

- dapat dievaluasi kembali sesuai rencana eksekusi;
- hanya berlaku untuk satu pernyataan;
- CTE yang terlalu banyak dapat membuat query tetap kompleks.

8. *CTE dan Subquery*

Aspek	Subquery	CTE
Penamaan Result Set	Biasanya menggunakan alias	Memiliki nama yang jelas
Keterbacaan	Dapat menurun jika bersarang	Umumnya lebih baik
Rekursi	Tidak dirancang untuk rekursi	Mendukung rekursi
Ruang lingkup	Bagian dari query	Satu pernyataan setelah definisi
Penggunaan bertahap	Lebih sulit	Lebih mudah

CTE tidak selalu lebih cepat daripada Subquery. Keunggulan utamanya adalah keterbacaan dan pengorganisasian yang logis.

F. CTE REKURSIF

1. *Pengertian CTE Rekursif*

CTE rekursif adalah CTE yang merujuk kembali pada dirinya sendiri. CTE rekursif digunakan untuk memproses:

- data hierarkis;
- struktur induk-anak;
- struktur organisasi;
- kategori bertingkat;
- silsilah;
- jalur hubungan;
- urutan nilai yang dibentuk berulang.

2. *Komponen CTE Rekursif*

CTE rekursif memiliki dua komponen utama.

a. Anchor member

Bagian awal yang menentukan titik awal proses.

b. Recursive member

Bagian yang mengembangkan hasil dari tingkat sebelumnya menuju tingkat berikutnya.

3. *Syarat Data Rekursif*

Agar CTE rekursif dapat diterapkan, struktur tabel harus memiliki hubungan hierarkis, misalnya:

KodeKategori

KodeKategoriInduk

NamaKategori

Pada struktur DB_Petshop saat ini, tabel KategoriProduk belum memiliki kolom kategori induk. Oleh karena itu, penggunaan CTE rekursif memerlukan pengembangan struktur tabel.

4. *Kondisi Berhenti*

Proses rekursif harus memiliki kondisi berhenti. Tanpa kondisi berhenti, proses dapat terus berulang. SQL Server memiliki pembatasan kedalaman rekursi untuk mencegah proses yang tak berujung. Namun, rancangan data dan logika tetap harus memastikan bahwa:

- tidak terdapat siklus;
- data induk tidak mengacu kembali kepada anak;
- setiap jalur memiliki akhir;
- relasi hierarki valid.

5. *Risiko CTE Rekursif*

- rekursi tanpa akhir;
- hubungan siklik;
- hasil duplikat;
- performa menurun;
- struktur sulit dipahami;
- tingkat hierarki terlalu dalam.

G. KONSEP DASAR WINDOW FUNCTION

1. *Pengertian Window Function*

Window Function adalah fungsi yang melakukan perhitungan terhadap sekumpulan baris yang memiliki hubungan dengan baris saat ini tanpa menggabungkan seluruh baris tersebut menjadi satu baris. Kata window menggambarkan sekumpulan baris yang membentuk area perhitungan.

Window Function dapat digunakan untuk:

- memberi nomor urut;
- menentukan peringkat;
- membagi data ke dalam kelompok;
- menghitung total berjalan;
- menghitung rata-rata bergerak;

- membandingkan baris saat ini dengan baris sebelumnya;
- Membandingkan baris saat ini dengan baris berikutnya.

2. *Perbedaan dengan Fungsi Agregasi*

Fungsi agregasi biasanya mengurangi jumlah baris.

Sebagai contoh konseptual:

Data awal

Pelanggan A — Transaksi 1

Pelanggan A — Transaksi 2

Pelanggan A — Transaksi 3

Setelah agregasi

Pelanggan A — Total transaksi

Window Function mempertahankan seluruh baris.

Pelanggan A — Transaksi 1 — Total pelanggan A

Pelanggan A — Transaksi 2 — Total pelanggan A

Pelanggan A — Transaksi 3 — Total pelanggan A

3. *Karakteristik Window Function*

- Tidak menghilangkan detail baris.
- Bekerja pada sekumpulan baris.
- Menggunakan klausa OVER.
- Dapat membagi data ke dalam beberapa partisi.
- Dapat mengurutkan data pada setiap partisi.
- Dapat menentukan jangkauan baris.
- Cocok untuk analisis berurutan dan peringkat.

4. *Jenis Window Function*

Window Function dapat dikelompokkan menjadi:

- Ranking functions
 - a. ROW_NUMBER
 - b. RANK
 - c. DENSE_RANK
 - d. NTILE
- Offset functions
 - a. LEAD
 - b. LAG

- c. FIRST_VALUE
- d. LAST_VALUE
- Aggregate window functions
 - a. SUM
 - b. COUNT
 - c. AVG
 - d. MIN
 - e. MAX

5. Penerapan pada DB_Petshop

Window Function dapat digunakan untuk:

- memberikan nomor urut transaksi;
- menentukan produk terlaris;
- menentukan peringkat pelanggan;
- mendapatkan transaksi terakhir setiap pelanggan;
- membagi pelanggan ke dalam empat kelompok;
- membandingkan penjualan dengan transaksi sebelumnya;
- membandingkan stok audit saat ini dengan audit sebelumnya;
- melihat reservasi berikutnya pada jadwal pegawai.

H. KLAUSA OVER

1. Pengertian OVER

OVER adalah klausa yang mendefinisikan window atau sekumpulan baris yang akan digunakan oleh Window Function. Tanpa OVER, fungsi seperti ROW_NUMBER, RANK, LEAD, dan LAG tidak memiliki konteks mengenai:

- kelompok data yang diproses;
- urutan baris;
- jangkauan perhitungan.

Modul ini berfokus pada ROW_NUMBER, RANK, DENSE_RANK, NTILE, LEAD, dan LAG.

2. Komponen OVER

Klausa OVER dapat terdiri atas:

- PARTITION BY
- ORDER BY
- Window frame

Struktur konseptual:



3. *PARTITION BY*

PARTITION BY membagi Result Set menjadi kelompok-kelompok yang lebih kecil. Setiap partisi diproses secara terpisah.

PARTITION BY dan *GROUP BY*

Aspek	<i>PARTITION BY</i>	<i>GROUP BY</i>
Mengelompokkan data	Ya	Ya
Mengurangi jumlah baris	Tidak	Ya
Mempertahankan detail	Ya	Tidak
Digunakan dalam	Window Function	Agregasi biasa

4. *ORDER BY* dalam *OVER*

ORDER BY di dalam *OVER* menentukan urutan analitis.

Urutan ini digunakan untuk:

- memberi nomor;
- menentukan peringkat;
- memilih baris sebelumnya;
- memilih baris berikutnya;
- menghitung total berjalan.

ORDER BY di dalam *OVER* berbeda dari pengurutan Result Set akhir.

5. *Window Frame*

Window frame menentukan bagian spesifik dari partisi yang digunakan dalam perhitungan. Jenis batas yang umum secara konseptual:

- dari awal partisi hingga baris saat ini;
- beberapa baris sebelum hingga baris saat ini;
- baris saat ini hingga beberapa baris berikutnya;
- seluruh partisi.

Window frame sangat relevan untuk:

- total berjalan;
- rata-rata bergerak;
- nilai minimum dan maksimum dalam periode tertentu.

I. ROW_NUMBER

1. Pengertian ROW_NUMBER

ROW_NUMBER adalah Window Function yang memberikan nomor urut unik pada setiap baris berdasarkan urutan tertentu. Nomor dimulai dari 1 dan terus bertambah 1 pada setiap baris.

Baris pertama → 1

Baris kedua → 2

Baris ketiga → 3

Baris keempat → 4

Tidak ada dua baris yang memiliki nomor yang sama dalam partisi yang sama.

2. Karakteristik ROW_NUMBER

- menghasilkan nomor unik;
- tidak mempertimbangkan nilai yang sama;
- tetap memberi nomor berbeda pada data seri;
- dapat dimulai ulang pada setiap partisi;
- memerlukan urutan analitis;
- cocok untuk pemilihan satu baris dari setiap kelompok.

3. Ilustrasi ROW_NUMBER

Misalkan terdapat total penjualan produk:

Produk	Total terjual
Produk A	100
Produk B	100
Produk C	80
Produk D	60

ROW_NUMBER menghasilkan:

Produk	Total terjual	Nomor
Produk A	100	1
Produk B	100	2
Produk C	80	3
Produk D	60	4

Walaupun Produk A dan Produk B memiliki nilai sama, keduanya mendapat nomor berbeda.

4. *Determinisme ROW_NUMBER*

Jika kolom pengurutan memiliki nilai yang sama, urutan antarbaris dapat menjadi tidak stabil. Agar hasil konsisten, diperlukan kolom tambahan yang unik sebagai penentu urutan.

J. RANK DAN DENSE_RANK

1. *Pengertian RANK*

RANK memberikan peringkat berdasarkan urutan nilai. Baris yang memiliki nilai yang sama memperoleh peringkat yang sama. Namun, setelah terjadi nilai seri, nomor peringkat berikutnya akan melompat.

Produk	Total	RANK
Produk A	100	1
Produk B	100	1
Produk C	80	3
Produk D	60	4

Peringkat dua tidak digunakan karena terdapat 2 baris pada peringkat pertama.

2. *Pengertian DENSE_RANK*

DENSE_RANK juga memberikan peringkat yang sama untuk nilai yang sama dalam satu seri. Perbedaannya, peringkat berikutnya tidak melompat.

Produk	Total	RANK
Produk A	100	1
Produk B	100	1
Produk C	80	2
Produk D	60	3

3. *Perbandingan ROW_NUMBER, RANK, dan DENSE_RANK*

Nilai	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
80	3	3	2
60	4	4	3

ROW_NUMBER

Setiap baris mendapat nomor berbeda.

RANK

Nilai yang sama mendapat peringkat yang sama dan menimbulkan lonjakan.

DENSE_RANK

Nilai sama mendapat peringkat sama tanpa lompatan.

4. *Kapan menggunakan RANK?*

RANK sesuai digunakan pada sistem peringkat kompetisi.

Contoh:

- dua produk berada pada peringkat pertama;
- produk berikutnya berada pada peringkat ketiga;
- Posisi kedua dianggap telah ditempati oleh data seri.

5. *Kapan menggunakan DENSE_RANK?*

DENSE_RANK cocok ketika pengguna membutuhkan daftar nilai tingkat tanpa celah.

Contoh:

- klasifikasi harga tertinggi pertama, kedua, dan ketiga;
- tingkat performa pegawai;
- urutan kelompok total pembelian;
- Daftar level stok.

6. *Penerapan pada DB_Petshop*

RANK

- peringkat produk berdasarkan jumlah terjual;
- peringkat pelanggan berdasarkan total pembelian;
- peringkat pegawai berdasarkan transaksi;
- Peringkat layanan berdasarkan jumlah reservasi.

DENSE_RANK

- kelompok harga produk;
- tingkat pendapatan pelanggan;
- kelompok performa pegawai;
- Tingkatan layanan berdasarkan popularitas.

K. NTILE

1. *Pengertian NTILE*

NTILE adalah Window Function yang membagi baris ke dalam sejumlah kelompok yang relatif seimbang. Misalnya, data pelanggan dibagi ke dalam empat kelompok:

Kelompok 1

Kelompok 2

Kelompok 3

Kelompok 4

NTILE tidak membagi berdasarkan rentang nilai yang tetap. NTILE membagi data berdasarkan jumlah baris setelah data diurutkan.

2. *Cara Kerja NTILE*

Jika terdapat 12 baris dan dibagi menjadi 4 kelompok:

Kelompok 1 → 3 baris

Kelompok 2 → 3 baris

Kelompok 3 → 3 baris

Kelompok 4 → 3 baris

Jika terdapat 10 baris dan dibagi menjadi 4 kelompok:

Kelompok 1 → 3 baris

Kelompok 2 → 3 baris

Kelompok 3 → 2 baris

Kelompok 4 → 2 baris

Kelompok awal memperoleh baris tambahan apabila pembagiannya tidak merata.

3. *Karakteristik NTILE*

- membagi data berdasarkan jumlah baris;
- memerlukan pengurutan;
- kelompok dibuat relatif seimbang;
- kelompok awal dapat memiliki satu baris lebih banyak;
- nilai yang sama dapat masuk ke kelompok berbeda;
- bukan pengelompokan berdasarkan batas nilai tetap.

4. *Penerapan NTILE pada DB_Petshop*

NTILE dapat digunakan untuk:

- membagi pelanggan menjadi empat kelompok berdasarkan total pembelian;
- membagi produk berdasarkan tingkat penjualan;

- membagi pegawai menjadi beberapa kelompok performa;
- membagi layanan berdasarkan popularitas;
- membentuk kelompok promosi;
- menentukan prioritas pengelolaan stok.

5. Keterbatasan NTILE

NTILE harus digunakan dengan hati-hati karena dua pelanggan dengan total pembelian yang sama dapat ditempatkan dalam kelompok yang berbeda jika batas kelompok berada di antara keduanya. Oleh karena itu, NTILE cocok untuk distribusi kuantitatif, tetapi belum tentu cocok untuk klasifikasi bisnis dengan batas nilai yang tegas.

L. LEAD DAN LAG

1. Konsep Offset Function

LEAD dan LAG termasuk dalam Offset Function, yaitu fungsi yang mengambil nilai dari baris lain berdasarkan posisi relatif terhadap baris saat ini.

Baris sebelumnya ← Baris saat ini → Baris berikutnya

LAG

LEAD

Fungsi tersebut memungkinkan perbandingan antarbaris tanpa melakukan SELF JOIN.

2. Pengertian LAG

LAG digunakan untuk mengambil nilai dari baris sebelumnya berdasarkan urutan yang ditentukan. Contoh kebutuhan:

- stok sebelum audit saat ini;
- nilai transaksi sebelumnya;
- tanggal reservasi sebelumnya;
- harga produk pada catatan sebelumnya;
- status sebelumnya.

3. Ilustrasi LAG

Tanggal	Nilai transaksi	Nilai sebelumnya
1 Januari	100	Tidak ada
2 Januari	150	100
3 Januari	120	150

Pada baris pertama, tidak terdapat baris sebelumnya sehingga nilai LAG biasanya NULL atau menggunakan nilai bawaan tertentu.

4. Pengertian LEAD

LEAD digunakan untuk mengambil nilai dari baris berikutnya.

Contoh kebutuhan:

- melihat jadwal reservasi berikutnya;
- melihat tanggal audit selanjutnya;
- membandingkan transaksi saat ini dengan transaksi berikutnya;
- menghitung jarak waktu ke aktivitas selanjutnya.

5. *Komponen LEAD dan LAG*

LEAD dan LAG secara konseptual memiliki tiga unsur:

- nilai yang akan diambil;
- jarak perpindahan;
- nilai bawaan jika baris tidak ditemukan.

Jarak perpindahan

Jarak satu berarti satu baris sebelum atau sesudah.

Jarak dua berarti dua baris sebelum atau sesudah.

Nilai bawaan

Nilai bawaan dapat digunakan sebagai pengganti NULL ketika baris tujuan tidak tersedia.